

Experiment-1

Creating a LAN using Packet Tracer

Aim: Connect the computers in Local Area Network.

Theory:

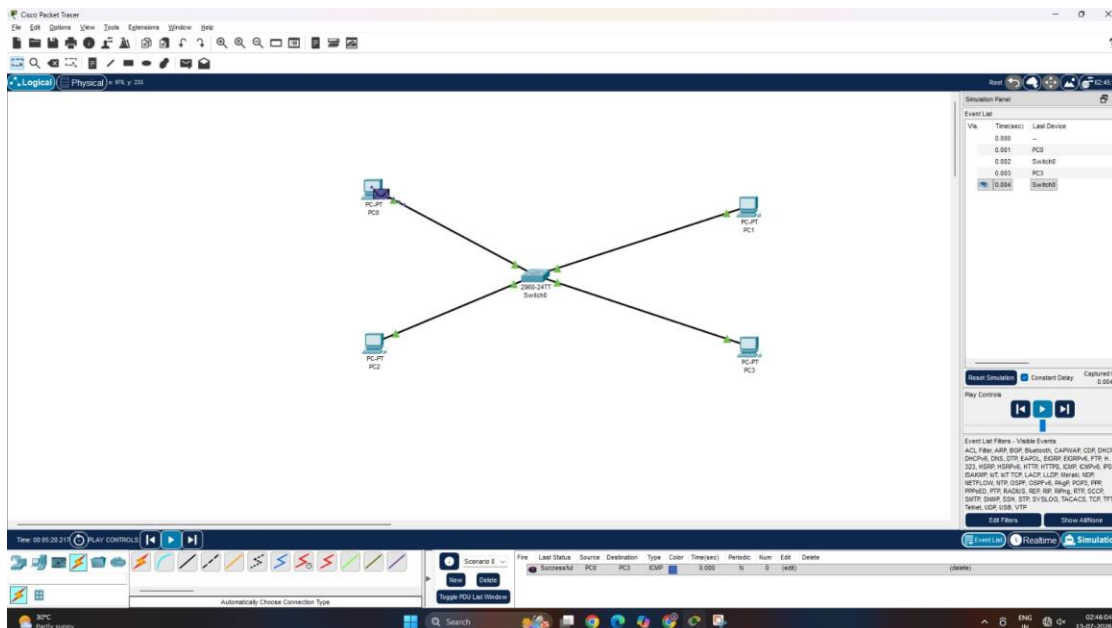
Copper straight-through: This is a standard Ethernet cable that is used to connect two devices that operate in different layers of the OSI model (such as hub to router and switch to PC).

Copper cross-over: This Ethernet cable connects devices operating in the same OSI layer (such as hub to hub, PC to PC, PC to router, and PC to printer).

Procedure:

1. Place 4 Computers on the workplace like the above diagram.
2. Add Switch at the centre.
3. Connect all the computers to the switch using Copper straight through cable.
4. Assign IP address to all the computers.
5. Add simple PDU and verify the design.

Network Diagram / Output:



OUTPUT:

The above program gets successfully executed.

Result:

The LAN was successfully created and connectivity was verified using Simple PDU in Cisco Packet Tracer.

Experiment-2

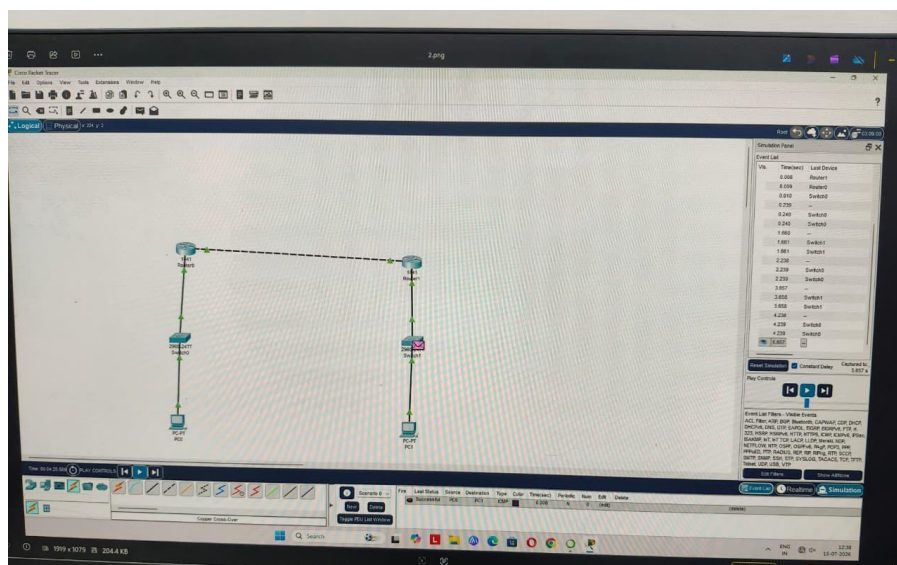
Experimenting with network protocols for achieving communication between computers using Packet Tracer

Aim: Connect the computers in Local Area Network and communicate using RIP (Routing Information Protocol).

Procedure:

1. Place two PC, Two switch and Two routers in workplace like above diagram.
2. Connect PC1 to the switch2 and PC2 to switch3 using Automatic connection.
3. Connect switch2 to the router0 and switch3 to router 1 using Automatic connection.
4. Connect Two routers using Automatic connection.
5. Set IP address and default gateway in both the PC.
6. In router0 set IP address in Fast Ethernet 0/0 and Fast Ethernet 0/1 (turn on).
7. In router0 add RIP address one after another using Add.
8. In router1 set IP address in Fast Ethernet 0/0 and Fast Ethernet 0/1 (turn on).
9. In router1 add RIP address one after another using Add.
10. Add simple PDU and verify the design.

Network Diagram / Output:



Result: The above program gets successfully executed. Communication between the computers was successfully achieved using RIP protocol.

Experiment-3

Interconnection software for communication between two different network architectures using Packet Tracer

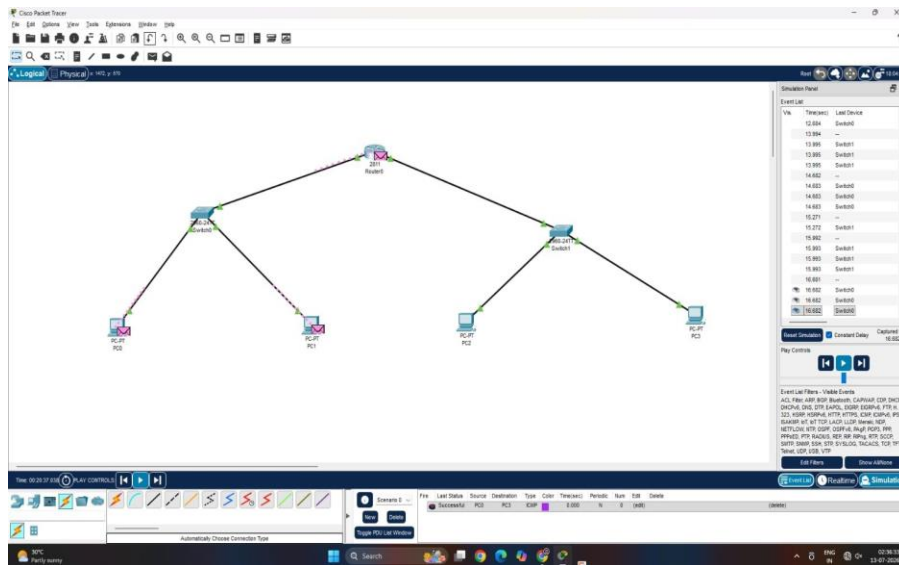
Aim:

Create two different network architectures and transmit the data between them.

Procedure:

1. Place 4 PC, 2 switches and 1 router in workplace like above diagram.
2. Connect below to above using Automatic connection.
3. Set the IP addresses (Different classes like Class A and Class C) and Default Gateways in all the PCs situated in two different LAN or Network.
4. Configure the Router Fast Ethernet 0/0 and 0/1 interface.
5. Add simple PDU and verify the design.

Network Diagram / Output:



OUTPUT: The above program gets successfully executed.

Result: Two different network architectures were successfully interconnected and data transmission was verified.

Experiment-4

Designing various topologies using Cisco Packet Tracer

Aim: Designing various topologies (Ring, Bus, Mesh and Star) using Cisco Packet Tracer.

Ring Topology

Ring Topology may be a network configuration where device connections create a circular data path. In this each device is connected to with its exactly two neighboring devices, like points on a circle which forms like a ring structure.

Advantages of Ring topology:

- Data flows in one direction which reduces the chance of packet collisions.
- Additional workstations can be added without impacting performance.
- Equal access to the resources.
- No need of server to control connectivity.
- Cheap to install and expand.
- Minimum collision.
- High speed.
- Easy to manage.

Disadvantages of Ring topology:

- Uni-directional ring – packet must pass through all nodes.
- If one workstation shuts down, entire network is affected.
- Slower than bus under some conditions.
- Expensive.
- Difficult to troubleshoot.

Star Topology

A star topology is a LAN configuration in which all nodes are individually connected to a central connection point such as a hub or switch.

Advantages of Star Topology:

- Very reliable – if one cable fails others still work.

- High-performing, no data collisions.
- Easy fault detection.
- Easy to install.

Disadvantages of Star Topology:

- Requires more cable.
- If hub fails entire network goes down.
- More expensive due to hub/switch.

Bus Topology

Bus topology connects each computer and network device to a single backbone cable. Data travels along the cable and each node checks the destination address.

Advantages of Bus Topology:

- Easiest topology. Works well for small networks.
- Less cable required.
- Cost-effective.
- Easy to expand.

Disadvantages of Bus Topology:

- Not suitable for large networks.
- Difficult to troubleshoot.
- Terminators required.
- If main cable fails whole network fails.

Mesh Topology:

In mesh topology every computer is interconnected to every other via dedicated links. There are $N(N-1)/2$ links for N nodes.

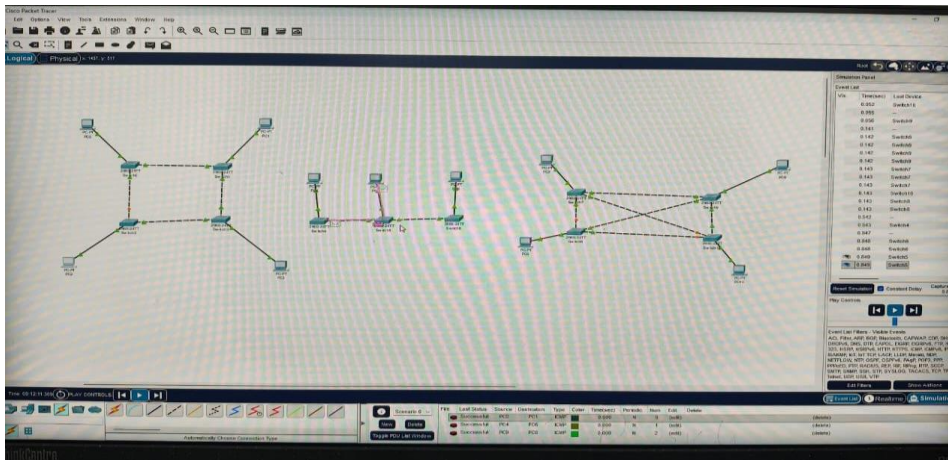
Advantages of Mesh Topology:

- Failure of single device does not break network.
- Dedicated links, high redundancy, high privacy and security.

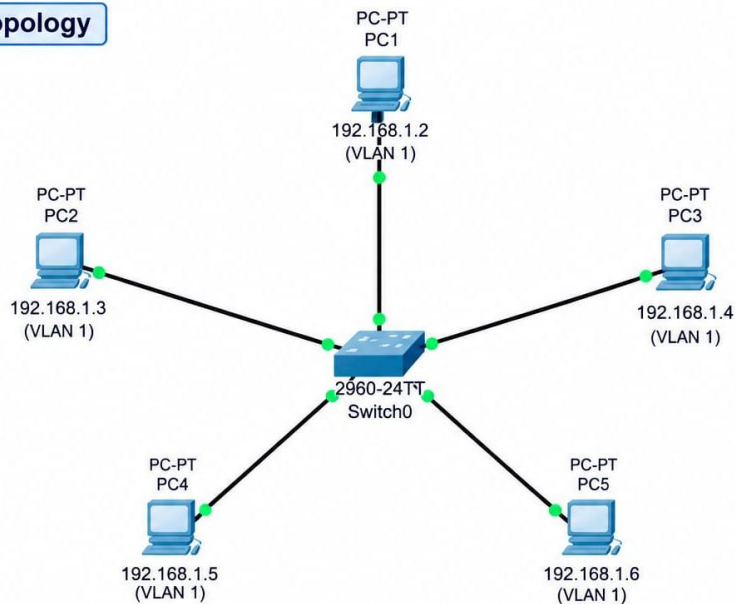
Disadvantages of Mesh Topology:

- Costly, difficult installation, high power requirement, complex maintenance.

Network Diagram / Output:



Star Topology



OUTPUT: The above program gets successfully executed.

Result:

Ring, Bus, Mesh and Star network topologies were successfully designed and verified using Cisco Packet Tracer.

Experiment-5

Case study: Sliding-Window Flow Control & Stop-And-Wait Flow Control

Aim:Case study on Sliding-Window Flow Control & Stop-And-Wait Flow Control.

Introduction:

Both Stop and Wait protocol and Sliding Window protocol are techniques for flow control. In Stop-and-Wait the sender sends one frame and waits for acknowledgement; in Sliding Window the sender can send more than one frame and retransmits damaged frames.

Stop-and-Wait Protocol

The sender transmits one packet at a time and waits for an acknowledgement before sending the next packet. It is simple but has low efficiency.

Sliding Window Protocol

Uses a window size to allow multiple packets to be transmitted without waiting for each acknowledgement. Higher efficiency; includes Go-Back-N and Selective Repeat.

Difference Table:

S.No	Stop-and-Wait Protocol	Sliding Window Protocol
1	Sender sends one frame and waits for ACK	Sender sends more than one frame; retransmits damaged frames
2	Efficiency is worse	Efficiency is better
3	Sender window size = 1	Sender window size = N
4	Receiver window size = 1	Receiver window size may be 1 or N
5	Efficiency = $1/(1+2*a)$	Efficiency = $N/(1+2*a)$
6	Half duplex	Full duplex

Conclusion:

Sliding Window protocol is more efficient and preferred for high-speed data transmission compared to Stop-and-Wait protocol.

Result:

The case study on Sliding-Window and Stop-and-Wait flow control protocols was successfully completed.

Experiment-6

Study of basic network commands

Aim: Verify basic network commands using command prompt.

Ping:

Ping is used to testing a network host capacity to interact with another host. Just enter the command Ping, followed by the target host's name or IP address. The ping utilities seem to be the most common network tool. This is performed by using the Internet Control Message Protocol, which allows the echo packet to be sent to the destination host and a listening mechanism. If the destination host reply to the requesting host that means the host is reachable. This utility usually gives a basic image of where there may be a specific networking issue

IpConfig:

The command IP config will display basic details about the device's IP address configuration. Just type IP config in the Windows prompt and the IP, subnet mask and default gateway that the current device will be presented. If you have to see full information, then type on command prompt config-all and then you will see full information. There are also choices to assist you in resolving DNS and DHCP issues.

Hostname :

To communicate with each and other, the computer needs a unique address. A hostname can be alphabetic or alphanumeric and contain specific symbols used specifically to define a specific node or device in the network. For example, a hostname should have a domain name (TLD) of the top-level and a distance between one and 63 characters when used in a domain name system (DNS) or on the Internet.

Tracert:

The tracert command is a Command Prompt command which is used to get the network packet being sent and received and the number of hops required for that packet to reach to target. This command can also be referred to as a traceroute. It provides several details about the path that a packet takes from the source to the specified destination. The tracert command is available for the Command Prompt in all Windows operating systems.

Nslookup :

The Nslookup, which stands for name server lookup command, is a network utility command used to obtain information about internet servers. It provides name server information for the DNS (Domain Name System), i.e. the default DNS server's name and IP Address.

ARP:

ARP stands for Address Resolution Protocol. Although network communications can readily be thought of as an IP address, the packet delivery depends ultimately on the media access control (MAC). This is where the protocol for address resolution comes into effect. You can add the remote host IP address, which is an arp -a command, in case you have issues to communicate with a given host. The ARP command provides information like Address, Hardware Type, Hardware Address, etc.

Ping

```
C:\Users\asus>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=11ms TTL=119
Reply from 8.8.8.8: bytes=32 time=8ms TTL=119
Reply from 8.8.8.8: bytes=32 time=35ms TTL=119

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 6ms, Maximum = 35ms, Average = 15ms
```

IpConfig:

```
C:\Users\asus>ipconfig

Windows IP Configuration

Wireless LAN adapter Local Area Connection* 9:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::cbf4:771d:3f7b:6e45%7
    IPv4 Address. . . . . : 172.16.19.243
    Subnet Mask . . . . . : 255.255.240.0
    Default Gateway . . . . . : 172.16.16.1

Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
```

Hostname

```
C:\Users\asus>hostname  
DESKTOP-G6F3CDS
```

Tracert

```
C:\Users\LENOVO>tracert  
Usage: tracert [-d] [-h maximum_hops] [-j host-list] [-w timeout]  
[-R] [-S srcaddr] [-4] [-6] target_name  
  
Options:  
-d Do not resolve addresses to hostnames.  
-h maximum_hops Maximum number of hops to search for target.  
-j host-list Loose source route along host-list (IPv4-only).  
-w timeout Wait timeout milliseconds for each reply.  
-R Trace round-trip path (IPv6-only).  
-S srcaddr Source address to use (IPv6-only).  
-4 Force using IPv4.  
-6 Force using IPv6.  
  
C:\Users\LENOVO>tracert 8.8.8.8  
Tracing route to dns.google [8.8.8.8]  
over a maximum of 30 hops:  
  1  139 ms  104 ms  312 ms  10.17.166.6  
  2         *      No resources.  
Trace complete.
```

Nslookup

```
C:\Users\LENOVO>nslookup  
Default Server: UnKnown  
Address: fec0:0:0:ffff::1
```

ARP

```
C:\Users\LENOVO>arp -a  
  
Interface: 192.168.137.1 --- 0x4  
Internet Address      Physical Address      Type  
192.168.137.255       ff-ff-ff-ff-ff-ff     static  
224.0.0.22            01-00-5e-00-00-16     static  
224.0.0.251           01-00-5e-00-00-fb     static  
224.0.0.252           01-00-5e-00-00-fc     static  
255.255.255.255       ff-ff-ff-ff-ff-ff     static  
  
C:\Users\LENOVO>
```

Observation Table

Command	Observation / Output
ping 8.8.8.8	Reply received; 0% loss; Avg RTT ~52 ms
ipconfig	IPv4 Address shown (e.g. 10.17.166.48); Subnet & Gateway displayed
hostname	DESKTOP-E09HM4U
tracert 8.8.8.8	Route traced; hops displayed
nslookup	Default Server and Address displayed
arp -a	ARP table with Internet & Physical addresses shown

Result:

All basic network commands were successfully executed and verified.

Experiment-7

Practice IPv4 and IPv6 Addressing, Subnetting, and Configure Static & Dynamic Routing (RIP/OSPF)

Aim:

To learn and implement IPv4 addressing and subnetting, IPv6 addressing, Static routing, Dynamic routing using RIP v2 and OSPF.

Objectives:

- Assign IPv4 and IPv6 addresses to routers and PCs.
- Design IPv4 subnets. Configure static routing, RIP version 2 and OSPF.
- Verify routing tables and end-to-end connectivity.

IP Addressing Table

Device	Interface	IPv4 Address	IPv6 Address
R1	G0/0	192.168.10.1/24	2001:DB8:1::1/64
R1	G0/1	10.0.12.1/30	2001:DB8:12::1/64
R2	G0/0	192.168.20.1/24	2001:DB8:2::1/64
R2	G0/1	10.0.12.2/30	2001:DB8:12::2/64
R2	G0/2	10.0.23.1/30	2001:DB8:23::1/64
R3	G0/0	192.168.30.1/24	2001:DB8:3::1/64
R3	G0/1	10.0.23.2/30	2001:DB8:23::2/64

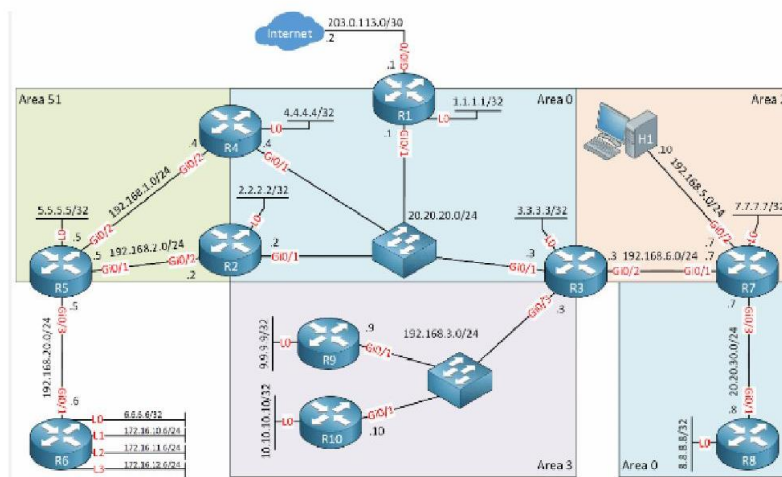
Procedure (Summary):

1. Create network with 3 routers and 3 PCs; connect with appropriate cables.
2. Configure IPv4 and IPv6 addresses on all interfaces and PCs.
3. Configure static routes, then RIP v2, then OSPF.
4. Verify with show ip route, show ip ospf neighbor, ping and traceroute.

IPv4 Subnetting Practice (192.168.1.0/24 into 4 equal subnets):

Subnet	First Host	Last Host	Broadcast
192.168.1.0/26	192.168.1.1	192.168.1.62	192.168.1.63
192.168.1.64/26	192.168.1.65	192.168.1.126	192.168.1.127
192.168.1.128/26	192.168.1.129	192.168.1.190	192.168.1.191
192.168.1.192/26	192.168.1.193	192.168.1.254	192.168.1.255

OUTPUT:



The Above Program gets successfully executed.

Result:

The network was successfully configured using IPv4 and IPv6 addressing. Static routing, RIP version 2 and OSPF were implemented successfully. End-to-end communication was verified.

Experiment-8

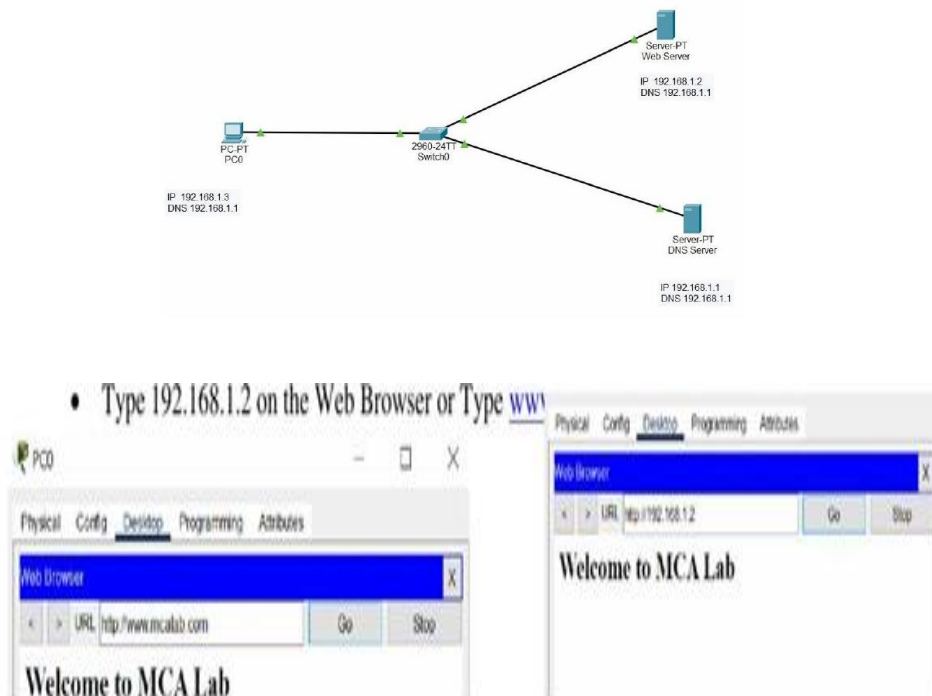
To configure Internet Access/Implementation using Cisco Packet Tracer

Aim: To configure Internet Access/Implementation using Cisco Packet Tracer.

Procedure:

- Connect the components as shown in the configuration.
- Rename server 1 as Web Server and server 2 as DNS Server.
- Set IP and DNS of Web Server; enable HTTP; edit index.html to: `<html><h1>Welcome to MCA Lab</h1></html>`
- On DNS server add resource record `www.mcalab.com` → `192.168.1.2`
- Configure PC IP; open web browser and type `192.168.1.2` or `www.mcalab.com`

Network Diagram / Output:



Result: The above program gets successfully executed. Internet/web access was successfully configured and tested.

Experiment-9

Error Detecting Code Using CRC-CCITT (16-bit)

Aim: Implement Error Detecting Code Using CRC-CCITT (16-bit) in C/C++.

Algorithm:

1. Start
2. Enter the message to be transmitted
3. Append 16 zeros for 16-bit CRC
4. XOR with polynomial 10001000000100001 and transmit
5. Verify received message
6. End

Code (key parts):

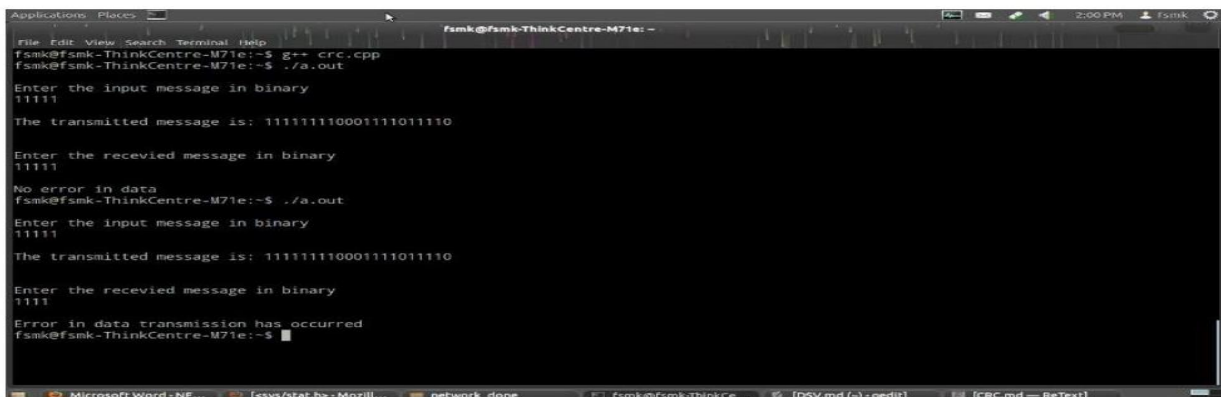
```
char poly[] = "10001000000100001";
```

```
crc(ip, op, poly, 1); // generate transmitted message
```

```
if (crc(recv, op, poly, 0)) cout << "No error in data"; else cout << "Error in data transmission has occurred";
```

Execution Commands:

- Open terminal → change to file location → g++ filename.cpp → ./a.out



```
fsmk@fsmk-ThinkCentre-M71e: ~  
fsmk@fsmk-ThinkCentre-M71e:~$ g++ crc.cpp  
fsmk@fsmk-ThinkCentre-M71e:~$ ./a.out  
Enter the input message in binary  
11111  
The transmitted message is: 11111111000111011110  
Enter the received message in binary  
11111  
No error in data  
fsmk@fsmk-ThinkCentre-M71e:~$ ./a.out  
Enter the input message in binary  
11111  
The transmitted message is: 11111111000111011110  
Enter the received message in binary  
11111  
Error in data transmission has occurred  
fsmk@fsmk-ThinkCentre-M71e:~$
```

Result: The CRC-CCITT (16-bit) error-detection program was successfully compiled and executed.

Experiment-10

Test HTTP/HTTPS Requests Using Postman and Browser Developer Tools

Aim:

To send and analyze HTTP/HTTPS requests using Postman and browser Developer Tools.

Objectives:

- To send HTTP GET/POST requests.
- To examine HTTP response status codes.
- To inspect request and response headers.
- To analyze network communication using browser Developer Tools.

Requirements:

- Computer with Internet connection, Postman, Web browser (Chrome/Edge/Firefox)

Procedure – A. Using Postman

1. Open Postman and create a new HTTP request.
2. Select GET or POST method.
3. Enter URL <https://httpbin.org/get> and click Send.
4. Observe status code, response time, body and headers.
5. Repeat with <https://www.google.com> and compare.

Observation Table — Postman

URL	Method	Status Code	Response Time	Result
https://httpbin.org/get	GET	200	~150 ms	Successful
https://www.google.com	GET	200	~80 ms	Successful

Procedure – B. Using Browser Developer Tools

1. Open browser and visit <https://www.google.com>
2. Press F12 → Network tab → Reload page
3. Select a request and examine URL, method, status, headers, response data and timing.

Observation Table — Browser Developer Tools

Parameter	Observation
Request URL	https://www.google.com/
Request Method	GET
Status Code	200
Content Type	text/html
Response Time	Observed in Timing tab
Request Headers	User-Agent, Accept, etc. present
Response Headers	Content-Type, Server, Date, etc. present

Result:

HTTP/HTTPS requests were successfully generated and analyzed using Postman and browser Developer Tools. The request methods, status codes, headers, response data and timing information were observed.

Experiment-11

Set Up FTP and Email (SMTP/POP3) Servers Using FileZilla and Thunderbird

Aim:

To set up and test an FTP server using FileZilla and configure an email client using Thunderbird to demonstrate SMTP and POP3 communication.

Part A: FTP Server using FileZilla

Procedure:

1. Install and open FileZilla Server; configure port 21.
2. Create user, set username/password, assign shared directory and permissions.
3. Start server. On client open FileZilla Client, enter Host, Username, Password, Port 21 → Quickconnect.
4. Upload and download a test file; verify transfer.

Observation Table – FTP

Parameter	Observation
FTP Port	21
Username	Configured user account
File Uploaded	Test file uploaded successfully
File Downloaded	Test file downloaded successfully
Transfer Status	Successful

Part B: Email using SMTP/POP3 with Thunderbird

Procedure:

1. Open Thunderbird → Add Mail Account.
2. Configure POP3 (incoming) and SMTP (outgoing) server addresses and ports.
3. Send a test email and retrieve it using POP3.

Observation Table – Email

Parameter	Observation
Email Protocols	SMTP for sending; POP3 for receiving
Test Email Sent	Yes
Test Email Received	Yes

Result:

The FTP server was successfully configured using FileZilla and files were transferred. SMTP was used for sending email and POP3 for receiving email using Thunderbird.

Experiment-12

Demonstrate Telnet and SSH Sessions to Remote Devices

Aim:

To establish remote sessions using Telnet and SSH and understand the differences between the two protocols.

Procedure – Telnet:

1. Enable Telnet service on the test server.
2. From client: telnet <IP-address> 23 → authenticate → execute commands → exit.

Procedure – SSH:

1. Enable SSH on remote system.
2. From client: ssh username@<IP-address> → authenticate → execute commands → exit.

Observation Table – Telnet and SSH

Parameter	Telnet	SSH
Default Port	23	22
Encryption	No	Yes
Authentication	Username/Password	Password/Keys
Security	Unencrypted	Encrypted
Remote Command Execution	Yes	Yes
Application	Legacy/testing	Secure remote administration

Result:

Telnet and SSH sessions were successfully established. The experiment demonstrated that SSH provides encrypted communication whereas Telnet does not.

Experiment-13

Create a Simple Web Service Client Using REST API (Python/Browser)

Aim: To create and test a simple REST API client using Python and a web browser.

Theory:

REST API uses HTTP methods (GET, POST, PUT, DELETE) and commonly returns JSON.

Procedure – Python:

1. Install requests library: pip install requests
2. Create rest_client.py with code that sends GET to <https://jsonplaceholder.typicode.com/posts/1>
3. Run: python rest_client.py and observe status code and JSON.

Program:

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url)

print("Status Code:", response.status_code)

print("Response Data:")

print(response.json())
```

Observation Table

Parameter	Observation
API URL	https://jsonplaceholder.typicode.com/posts/1
HTTP Method	GET
Status Code	200
Response Format	JSON
Response Data	{'userId': 1, 'id': 1, 'title': '...', 'body': '...'}

Result: A simple REST API client was successfully created using Python. The REST API response was retrieved and analyzed using Python and a web browser.

Experiment-14

Configure Basic Firewall Rules Using UFW

Aim:

To configure and test basic firewall rules using UFW (Uncomplicated Firewall).

Procedure:

1. Check status: `sudo ufw status`
2. Enable: `sudo ufw enable`
3. Allow SSH, HTTP, HTTPS: `sudo ufw allow 22/tcp ; sudo ufw allow 80/tcp ; sudo ufw allow 443/tcp`
4. Deny test port: `sudo ufw deny 8080/tcp`
5. Display rules: `sudo ufw status numbered`

Observation Table

Rule	Port	Protocol	Action
SSH	22	TCP	Allow
HTTP	80	TCP	Allow
HTTPS	443	TCP	Allow
Test Service	8080	TCP	Deny

Result:

Basic firewall rules were successfully configured and verified using UFW.

Experiment-15

Capture WPA2 Authentication Traffic Using Wireshark

Aim:

To capture and analyze WPA2 authentication traffic using Wireshark in an authorized wireless laboratory environment.

Important:

Perform this experiment only on a wireless network that you own or have explicit permission to monitor. Limited to passive capture; do not attempt password cracking or deauthentication attacks.

Theory:

WPA2 uses the 4-way handshake (four EAPOL-Key messages) to establish session keys between client and access point.

Procedure:

1. Prepare dedicated WPA2-Personal test network and connect authorized client.
2. Open Wireshark, select wireless interface, start capture.
3. Reconnect client and allow authentication to complete; stop capture.
4. Apply filter: eapol and examine the 4-way handshake frames.

Observation Table

Parameter	Observation
Wireless Network (SSID)	WPA2 Test Network
Security	WPA2
EAPOL Frames Captured	4
Handshake Messages Observed	4-way handshake (Msg. 1, 2, 3, 4)

Result:

WPA2 authentication traffic was successfully captured in an authorized laboratory environment. The EAPOL frames associated with the WPA2 4-way handshake were identified and analyzed using Wireshark.